

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 40%

QUESTIONS: 2

Duration : 45 Minutes
Total Marks:20

Annexure A

INTEGRATED EXPERIMENTS

Code of Conduct:

I A G Jayapradhan (192471016) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



INTEGRATED EXPERIMENTS

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

- (a) Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.
- (b) Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification
- (c) Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

- (a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, -1 each step, -5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α , γ , ϵ).
- (b) Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.
- (c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

INTEGRATED EXPERIMENTS – ASSESSMENT

Experiment 1: Decision Tree Classification | Experiment 2: Reinforcement Learning – Q-Learning

Experiment 1 – Decision Tree Classification

1. Aim

To implement and evaluate a Decision Tree classifier on the Iris dataset using Python, using Information Gain (Entropy) as the splitting criterion, and to analyze the resulting classification performance.

2. Dataset Description

The Iris dataset contains 150 observations belonging to three classes: Iris-setosa, Iris-versicolor, and Iris-virginica. Each observation has four numerical attributes: sepal length, sepal width, petal length, and petal width. The dataset has no missing values, so no imputation is required.

3. Procedure

Load the Iris dataset and convert it into a DataFrame.

Check the first five rows, shape, data types, and missing values.

Split the data into 80% training data and 20% testing data using stratified sampling.

Build a Decision Tree using criterion='entropy'.

Train the model and display the learned tree structure.

Identify the root feature using the highest information gain.

Predict the test samples and calculate accuracy, confusion matrix, and classification report.

Identify and discuss the misclassified test instances.

4. Python Program

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

```

# Load dataset
iris = load_iris(as_frame=True)
df = iris.frame

print("First 5 rows:")
print(df.head())
print("\nData types:")
print(df.dtypes)
print("\nMissing values:")
print(df.isnull().sum())

X = iris.data
y = iris.target

# 80:20 stratified split
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.20, random_state=42, stratify=y
)

# Decision Tree using Information Gain / Entropy
model = DecisionTreeClassifier(
criterion="entropy", random_state=42, max_depth=3
)
model.fit(X_train, y_train)

# Tree structure
tree = export_text(model, feature_names=list(X.columns), decimals=2)
print("\nDecision Tree:\n", tree)

# Prediction and evaluation
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n",
classification_report(y_test, y_pred,
target_names=iris.target_names))

# Misclassified instances
for i, (actual, predicted) in enumerate(zip(y_test, y_pred)):
if actual != predicted:
print("Misclassified:", X_test.iloc[i].to_dict(),
"Actual:", iris.target_names[actual],
"Predicted:", iris.target_names[predicted])

```

5. Output / Results

Training samples: 120 Testing samples: 30

Accuracy: 96.67%

Confusion Matrix:

10 0 0

0 9 1

0 0 10

Classification Report:

precision recall f1-score support

setosa 1.00 1.00 1.00 10

versicolor 1.00 0.90 0.95 10

virginica 0.91 1.00 0.95 10

accuracy 0.97 30

macro avg 0.97 0.97 0.97 30

weighted avg 0.97 0.97 0.97 30

Learned Decision Tree:

```
|--- petal length (cm) <= 2.45
| |--- class: 0
|--- petal length (cm) > 2.45
| |--- petal width (cm) <= 1.65
| | |--- petal length (cm) <= 4.95
| | | |--- class: 1
| | |--- petal length (cm) > 4.95
| | | |--- class: 2
| |--- petal width (cm) > 1.65
| | |--- petal length (cm) <= 4.85
| | | |--- class: 2
| | |--- petal length (cm) > 4.85
| | | |--- class: 2
```

Root-node justification: With entropy as the criterion, the feature selected at the root is the one that provides the greatest reduction in class impurity (highest information gain). In this run, the root split is based on petal width.

6. Misclassified Instances

1. Features = {'sepal length (cm)': 6.7, 'sepal width (cm)': 3.0, 'petal length (cm)': 5.0, 'petal width (cm)': 1.7};
Actual = versicolor; Predicted = virginica.

7. Performance Analysis

The Decision Tree achieved an accuracy of 96.67% on the test set. The confusion matrix shows that the three Iris classes are separated very effectively, especially Iris-setosa. Any errors occur between the two more similar classes, versicolor and virginica. This demonstrates that Decision Trees can learn useful non-linear decision boundaries from numerical features.

8. Result

The Decision Tree classifier was successfully implemented using the entropy criterion. The model was trained, its tree structure was examined, and its classification performance was evaluated using accuracy, confusion matrix, and classification report.

Integrated Experiments – Assessment

Experiment 2 – Reinforcement Learning: Q-Learning

1. Aim

To implement Q-Learning for a simple 4×4 Grid World, observe the evolution of Q-values during training, extract the final optimal policy, and analyze cumulative reward convergence.

2. Environment Definition

The environment is a 4×4 grid. The agent starts at (0,0) and must reach the goal at (3,3). The obstacle is placed at (1,1). The available actions are Up, Down, Left, and Right.

Element	Definition	Value
States	Grid positions	16 states
Actions	Movement directions	Up, Down, Left, Right
Goal	Terminal state	(3,3)
Obstacle	Terminal penalty state	(1,1)
Normal move	Reward	-1
Goal	Reward	+10
Obstacle	Reward	-5
Learning rate α	Step size	0.10
Discount γ	Future reward weight	0.90
Initial ϵ	Exploration probability	0.20

3. Q-Learning Formula

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

The Q-table is initialized with zeros. During each episode, the agent selects actions using an ϵ -greedy strategy, receives a reward, and updates the Q-value using the above equation.

4. Python Program

```
import numpy as np
import random
import matplotlib.pyplot as plt

N = 4
states = [(r, c) for r in range(N) for c in range(N)]
actions = [0, 1, 2, 3] # Up, Down, Left, Right
goal = (3, 3)
obstacle = (1, 1)
start = (0, 0)

alpha = 0.10
gamma = 0.90
epsilon = 0.20
episodes = 100

Q = {s: np.zeros(4) for s in states}
episode_rewards = []

def step(state, action):
    r, c = state
    nr, nc = r, c

    if action == 0: nr -= 1
    elif action == 1: nr += 1
    elif action == 2: nc -= 1
    else: nc += 1

    # Boundary handling
    if nr < 0 or nr >= N or nc < 0 or nc >= N:
        return state, -1, False

    new_state = (nr, nc)

    if new_state == obstacle:
        return new_state, -5, True
    if new_state == goal:
        return new_state, +10, True

    return new_state, -1, False

for episode in range(1, episodes + 1):
    state = start
    total_reward = 0

    # Gradually reduce exploration
    eps = max(0.02, epsilon * (0.98 ** (episode - 1)))

    for _ in range(100):
        if random.random() < eps:
            action = random.choice(actions)
        else:
            action = np.argmax(Q[state])

        next_state, reward, done = step(state, action)

        best_next = 0 if done else np.max(Q[next_state])
        Q[state][action] += alpha * (
            reward + gamma * best_next - Q[state][action]
```

```

)

total_reward += reward
state = next_state

if done:
    break

episode_rewards.append(total_reward)

if episode in [1, 50, 100]:
    print("Episode", episode)
    print("Q(0,0) =", np.round(Q[(0,0)], 2))
    print("Q(2,2) =", np.round(Q[(2,2)], 2))

# Final policy
arrows = ["↑", "↓", "←", "→"]
print("\nFinal Learned Policy:")
for r in range(N):
    row = []
    for c in range(N):
        s = (r, c)
        if s == goal:
            row.append("G")
        elif s == obstacle:
            row.append("X")
        else:
            row.append(arrows[np.argmax(Q[s])])
    print(row)

# Cumulative reward plot
plt.plot(episode_rewards)
plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")
plt.title("Q-Learning: Reward per Episode")
plt.grid()
plt.show()

```

5. Q-Value Evolution

Episode 1: $Q(0,0) = [-0.27, -0.10, 0.00, 0.00]$ | $Q(2,2) = [0.00, 0.00, 0.00, 0.00]$
 Episode 50: $Q(0,0) = [-2.12, -2.13, -2.13, -2.11]$ | $Q(2,2) = [-0.36, 4.62, -0.11, 0.23]$
 Episode 100: $Q(0,0) = [-2.28, 0.23, -2.29, -2.24]$ | $Q(2,2) = [-0.36, 5.71, -0.11, 0.23]$

6. Final Learned Policy

```

↓ → ↓ ↓
↓ - ↓ ↓
↓ ↓ ↓ ↓
→ → → -

```

Legend: ↑ = Up, ↓ = Down, ← = Left, → = Right, X = obstacle, G = goal.

The learned policy generally directs the agent toward the goal while avoiding the obstacle. Because Q-Learning uses exploration, early episodes may contain inefficient moves and negative rewards. As the Q-values are updated, the agent increasingly selects actions with higher expected future return.

7. Convergence Behaviour

At the beginning of training, the Q-table contains zeros and the agent has no knowledge of the environment. Consequently, the cumulative reward varies considerably. With repeated interaction, useful values propagate back from the goal state to earlier states. The reward curve is expected to become more stable as exploration decreases and the agent follows the learned policy more often. Minor fluctuations can remain because the ϵ -greedy strategy still permits exploration.

8. Result

Q-Learning was successfully implemented for the 4×4 Grid World. The Q-table evolved through repeated episodes, the optimal action for each reachable state was extracted as the final policy, and cumulative reward was used to study learning and convergence.

Overall Conclusion

The two experiments demonstrate different forms of intelligent decision-making. Decision Tree Classification learns a supervised mapping from labeled data to class predictions using information gain, whereas Q-Learning learns an action policy through trial-and-error interaction with an environment. The experiments show how machine learning can be applied both to prediction problems and sequential decision-making problems.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation